

Suffix Arrays I

Paweł Gawrychowski

University of Wrocław

Recall that we assume the existence of the following device:

Suffix array + constant time RMQ queries

Given a text $w[1..n]$, we can construct in linear time a structure of size $\mathcal{O}(n)$ which allows us to answer any query of the form “what is the longest common prefix of $w[i..n]$ and $w[j..n]$?” in constant time.

During the next two lectures we will see how this can be done.

Recall that we assume the existence of the following device:

Suffix array + constant time RMQ queries

Given a text $w[1..n]$, we can construct in linear time a structure of size $\mathcal{O}(n)$ which allows us to answer any query of the form “what is the longest common prefix of $w[i..n]$ and $w[j..n]$?” in constant time.

During the next two lectures we will see how this can be done.

Recall that we assume the existence of the following device:

Suffix array + constant time RMQ queries

Given a text $w[1..n]$, we can construct in linear time a structure of size $\mathcal{O}(n)$ which allows us to answer any query of the form “what is the longest common prefix of $w[i..n]$ and $w[j..n]$?” in constant time.

During the next two lectures we will see how this can be done.

Goal

We already know how to solve the pattern matching problems: given a pattern and a text we can find all occurrences in linear time. But what if the text doesn't change, and we need to answer **MANY** queries about the occurrences of different patterns?

This is known as text indexing, and can be solved using suffix arrays. Later we will see other solutions, which can be modified to work even for dynamically changing text (think about appending characters to a very long document...).

Goal

We already know how to solve the pattern matching problems: given a pattern and a text we can find all occurrences in linear time. But what if the text doesn't change, and we need to answer **MANY** queries about the occurrences of different patterns?

This is known as text indexing, and can be solved using suffix arrays. Later we will see other solutions, which can be modified to work even for dynamically changing text (think about appending characters to a very long document...).

Lexicographical comparison

$s \leq t$ if either s is a prefix of t , or s and t agree on the first $i - 1$ positions, i.e., $s[1] = t[1]$, $s[2] = t[2]$, ..., $s[i - 1] = t[i - 1]$, and then $s[i] < t[i]$.

Compare *bbab* with *bbb*. Then compare *aab* with *aab*.

One can check that using the character $\leq$ makes sense: the relation is a total order (verify this!).

Lexicographical comparison

$s \leq t$ if either s is a prefix of t , or s and t agree on the first $i - 1$ positions, i.e., $s[1] = t[1]$, $s[2] = t[2]$, ..., $s[i - 1] = t[i - 1]$, and then $s[i] < t[i]$.

Compare *bbab* with *bbb*. Then compare *aab* with *aab*.

One can check that using the character $\leq$ makes sense: the relation is a total order (verify this!).

Lexicographical comparison

$s \leq t$ if either s is a prefix of t , or s and t agree on the first $i - 1$ positions, i.e., $s[1] = t[1]$, $s[2] = t[2]$, ..., $s[i - 1] = t[i - 1]$, and then $s[i] < t[i]$.

Compare *bbab* with *bbb*. Then compare *aab* with *aab*.

One can check that using the character $\leq$ makes sense: the relation is a total order (verify this!).

Now the suffix array is simply the lexicographically sorted list of all suffixes of a given word w .

$w = \text{mississippi}$

$SA[1] =$	11	=	i
$SA[2] =$	8	=	ippi
$SA[3] =$	5	=	issippi
$SA[4] =$	2	=	ississippi
$SA[5] =$	1	=	mississippi
$SA[6] =$	10	=	pi
$SA[7] =$	9	=	ppi
$SA[8] =$	7	=	sippi
$SA[9] =$	4	=	sissippi
$SA[10] =$	6	=	ssippi
$SA[11] =$	3	=	ssissippi

Why this is useful?

Generating all occurrences of a given pattern

Say that we want to output all occurrences of $p = \text{ippi}$. How do they look when we look at the table?

Lemma

All occurrences of the same p create a contiguous fragment $SA[i], SA[i + 1], \dots, SA[j]$ of the suffix array.

Proof: look at the whiteboard.

The question is how to determine this fragment?

Why this is useful?

Generating all occurrences of a given pattern

Say that we want to output all occurrences of $p = \text{ippi}$. How do they look when we look at the table?

Lemma

All occurrences of the same p create a contiguous fragment $SA[i], SA[i + 1], \dots, SA[j]$ of the suffix array.

Proof: look at the whiteboard.

The question is how to determine this fragment?

Why this is useful?

Generating all occurrences of a given pattern

Say that we want to output all occurrences of $p = \text{ippi}$. How do they look when we look at the table?

Lemma

All occurrences of the same p create a contiguous fragment $SA[i], SA[i + 1], \dots, SA[j]$ of the suffix array.

Proof: look at the whiteboard.

The question is how to determine this fragment?

Why this is useful?

Generating all occurrences of a given pattern

Say that we want to output all occurrences of $p = \text{ippi}$. How do they look when we look at the table?

Lemma

All occurrences of the same p create a contiguous fragment $SA[i], SA[i + 1], \dots, SA[j]$ of the suffix array.

Proof: look at the whiteboard.

The question is how to determine this fragment?

Lemma

$$SA[i - 1] < p \leq SA[i].$$

Proof: look at the whiteboard.

Recall that $SA[1] < SA[2] < \dots < SA[n]$. Hence we can binary search for the value of i ! And then verify if it corresponds to an occurrence, i.e., whether p is indeed a prefix of $SA[i]$.

How to determine j ?

Lemma

$$SA[i - 1] < p \leq SA[i].$$

Proof: look at the whiteboard.

Recall that $SA[1] < SA[2] < \dots < SA[n]$. Hence we can binary search for the value of i ! And then verify if it corresponds to an occurrence, i.e., whether p is indeed a prefix of $SA[i]$.

How to determine j ?

So far so good, but there are at least three questions:

- How much time the binary search takes?
- How much space do we need to store the suffix array?
- How much time do we need to compute the suffix array?

So far so good, but there are at least three questions:

- How much time the binary search takes?
- How much space do we need to store the suffix array?
- How much time do we need to compute the suffix array?

So far so good, but there are at least three questions:

- How much time the binary search takes?
- How much space do we need to store the suffix array?
- How much time do we need to compute the suffix array?

So far so good, but there are at least three questions:

- How much time the binary search takes?
- How much space do we need to store the suffix array?
- How much time do we need to compute the suffix array?

Storing the suffix array is easy: even though we think that $SA[i]$ is a word, it is really just a number denoting the starting position in w . Having the number we can access any letter of the word corresponding to $SA[i]$ in constant time. Hence the required space is $\mathcal{O}(n)$.

Binary searching is more tricky. There are just $\mathcal{O}(\log n)$ iterations, but each of them requires...

... $\mathcal{O}(m)$ time. Hence the total complexity is $\mathcal{O}(m \log n)$. Later we will see how to decrease it to $\mathcal{O}(m + \log n)$.

Finally, constructing the suffix array in a naive way would take $\mathcal{O}(n^2 \log n)$ time. Later we will see how to decrease it to $\mathcal{O}(n)$.

Storing the suffix array is easy: even though we think that $SA[i]$ is a word, it is really just a number denoting the starting position in w . Having the number we can access any letter of the word corresponding to $SA[i]$ in constant time. Hence the required space is $\mathcal{O}(n)$.

Binary searching is more tricky. There are just $\mathcal{O}(\log n)$ iterations, but each of them requires...

... $\mathcal{O}(m)$ time. Hence the total complexity is $\mathcal{O}(m \log n)$. Later we will see how to decrease it to $\mathcal{O}(m + \log n)$.

Finally, constructing the suffix array in a naive way would take $\mathcal{O}(n^2 \log n)$ time. Later we will see how to decrease it to $\mathcal{O}(n)$.

Storing the suffix array is easy: even though we think that $SA[i]$ is a word, it is really just a number denoting the starting position in w . Having the number we can access any letter of the word corresponding to $SA[i]$ in constant time. Hence the required space is $\mathcal{O}(n)$.

Binary searching is more tricky. There are just $\mathcal{O}(\log n)$ iterations, but each of them requires...

... $\mathcal{O}(m)$ time. Hence the total complexity is $\mathcal{O}(m \log n)$. Later we will see how to decrease it to $\mathcal{O}(m + \log n)$.

Finally, constructing the suffix array in a naive way would take $\mathcal{O}(n^2 \log n)$ time. Later we will see how to decrease it to $\mathcal{O}(n)$.

Storing the suffix array is easy: even though we think that $SA[i]$ is a word, it is really just a number denoting the starting position in w . Having the number we can access any letter of the word corresponding to $SA[i]$ in constant time. Hence the required space is $\mathcal{O}(n)$.

Binary searching is more tricky. There are just $\mathcal{O}(\log n)$ iterations, but each of them requires...

... $\mathcal{O}(m)$ time. Hence the total complexity is $\mathcal{O}(m \log n)$. Later we will see how to decrease it to $\mathcal{O}(m + \log n)$.

Finally, constructing the suffix array in a naive way would take $\mathcal{O}(n^2 \log n)$ time. Later we will see how to decrease it to $\mathcal{O}(n)$.

For the linear time suffix array construction algorithm we will need two basic building blocks.

Radix sort

A sequence of n numbers from $\{1, 2, \dots, k\}$ can be sorted in $\mathcal{O}(n + k)$ time. A sequence of n pairs from $\{1, \dots, k\} \times \{1, \dots, k\}$ can be sorted in the same complexity.

What about a sequence of triples?

Merging

Two sorted sequences $a_1, a_2, \dots, a_n$ and $b_1, b_2, \dots, b_m$ can be merged in $\mathcal{O}(n + m)$ time.

For the linear time suffix array construction algorithm we will need two basic building blocks.

Radix sort

A sequence of n numbers from $\{1, 2, \dots, k\}$ can be sorted in $\mathcal{O}(n + k)$ time. A sequence of n pairs from $\{1, \dots, k\} \times \{1, \dots, k\}$ can be sorted in the same complexity.

What about a sequence of triples?

Merging

Two sorted sequences $a_1, a_2, \dots, a_n$ and $b_1, b_2, \dots, b_m$ can be merged in $\mathcal{O}(n + m)$ time.

For the linear time suffix array construction algorithm we will need two basic building blocks.

Radix sort

A sequence of n numbers from $\{1, 2, \dots, k\}$ can be sorted in $\mathcal{O}(n + k)$ time. A sequence of n pairs from $\{1, \dots, k\} \times \{1, \dots, k\}$ can be sorted in the same complexity.

What about a sequence of triples?

Merging

Two sorted sequences $a_1, a_2, \dots, a_n$ and $b_1, b_2, \dots, b_m$ can be merged in $\mathcal{O}(n + m)$ time.

For the linear time suffix array construction algorithm we will need two basic building blocks.

Radix sort

A sequence of n numbers from $\{1, 2, \dots, k\}$ can be sorted in $\mathcal{O}(n + k)$ time. A sequence of n pairs from $\{1, \dots, k\} \times \{1, \dots, k\}$ can be sorted in the same complexity.

What about a sequence of triples?

Merging

Two sorted sequences $a_1, a_2, \dots, a_n$ and $b_1, b_2, \dots, b_m$ can be merged in $\mathcal{O}(n + m)$ time.

So far we did not care about the size of the alphabet that much. For the reason that should be clear after the next hour, today we will to work with a word w where each letter is an integer from $\{1, 2, \dots, |w|\}$.

The goal...

is to develop an $\mathcal{O}(n)$ time algorithm, where $n = |w|$.

So far we did not care about the size of the alphabet that much. For the reason that should be clear after the next hour, today we will to work with a word w where each letter is an integer from $\{1, 2, \dots, |w|\}$.

The goal...

is to develop an $\mathcal{O}(n)$ time algorithm, where $n = |w|$.

Let's start with an $\mathcal{O}(n^2)$ time algorithm.

Radix sort

A sequence of n pairs from $\{1, \dots, n\} \times \{1, \dots, n\}$ can be sorted in $\mathcal{O}(n)$ time.

$\mathcal{O}(n^2)$ algorithm

For each $i = n, n-1, n-2, \dots, 1$ construct a sorted list L_i containing all $w[i..n]$, $w[i+1..n]$, $w[i+2..n]$, ... $w[n..n]$.

Assume that we have the list L_{i+1} , and want to construct the list for i . For each $j = i, i+1, \dots, n$ construct a pair $(w[j], \text{nr}_{i+1}[j+1])$, where $\text{nr}_i(j)$ is the position of $w[j..n]$ on the list L_i . Then sort all pairs, and notice that their order determines L_{i+1} .

Let's start with an $\mathcal{O}(n^2)$ time algorithm.

Radix sort

A sequence of n pairs from $\{1, \dots, n\} \times \{1, \dots, n\}$ can be sorted in $\mathcal{O}(n)$ time.

$\mathcal{O}(n^2)$ algorithm

For each $i = n, n-1, n-2, \dots, 1$ construct a sorted list L_i containing all $w[i..n]$, $w[i+1..n]$, $w[i+2..n]$, ... $w[n..n]$.

Assume that we have the list L_{i+1} , and want to construct the list for i . For each $j = i, i+1, \dots, n$ construct a pair $(w[j], \text{nr}_{i+1}[j+1])$, where $\text{nr}_i(j)$ is the position of $w[j..n]$ on the list L_i . Then sort all pairs, and notice that their order determines L_{i+1} .

Let's start with an $\mathcal{O}(n^2)$ time algorithm.

Radix sort

A sequence of n pairs from $\{1, \dots, n\} \times \{1, \dots, n\}$ can be sorted in $\mathcal{O}(n)$ time.

$\mathcal{O}(n^2)$ algorithm

For each $i = n, n-1, n-2, \dots, 1$ construct a sorted list L_i containing all $w[i..n]$, $w[i+1..n]$, $w[i+2..n]$, ... $w[n..n]$.

Assume that we have the list L_{i+1} , and want to construct the list for i . For each $j = i, i+1, \dots, n$ construct a pair $(w[j], \text{nr}_{i+1}[j+1])$, where $\text{nr}_i(j)$ is the position of $w[j..n]$ on the list L_i . Then sort all pairs, and notice that their order determines L_{i+1} .

Let's start with an $\mathcal{O}(n^2)$ time algorithm.

Radix sort

A sequence of n pairs from $\{1, \dots, n\} \times \{1, \dots, n\}$ can be sorted in $\mathcal{O}(n)$ time.

$\mathcal{O}(n^2)$ algorithm

For each $i = n, n-1, n-2, \dots, 1$ construct a sorted list L_i containing all $w[i..n]$, $w[i+1..n]$, $w[i+2..n]$, ... $w[n..n]$.

Assume that we have the list L_{i+1} , and want to construct the list for i . For each $j = i, i+1, \dots, n$ construct a pair $(w[j], \text{nr}_{i+1}[j+1])$, where $\text{nr}_i(j)$ is the position of $w[j..n]$ on the list L_i . Then sort all pairs, and notice that their order determines L_{i+1} .

Now let's try to improve the complexity to $\mathcal{O}(n \log n)$.

When you think about the previous solution, some fragments of the word will be compared again... and again...

We apply the *doubling* paradigm. To make the description easier, append n additional null characters to w , so it's actually a word of length $2n$.

$\mathcal{O}(n \log n)$ algorithm

For each $i = 0, 1, 2, \dots, \log n$ construct a sorted list L_i containing all $w[1..1 + 2^i - 1]$, $w[2..2 + 2^i - 1]$, ..., $w[n..n + 2^i - 1]$.

Assume that we have the list L_i , and want to construct the list for $i + 1$. For each $j = 1, 2, \dots, n$ construct a pair $(\text{nr}_i(j), \text{nr}_i(j + 2^i))$, where $\text{nr}_i(j)$ is the position of $w[j..j + 2^i - 1]$ on the list L_i . Then sort all pairs, and notice that their order determines L_{i+1} .

Now let's try to improve the complexity to $\mathcal{O}(n \log n)$.

When you think about the previous solution, some fragments of the word will be compared again... and again...

We apply the *doubling* paradigm. To make the description easier, append n additional null characters to w , so it's actually a word of length $2n$.

$\mathcal{O}(n \log n)$ algorithm

For each $i = 0, 1, 2, \dots, \log n$ construct a sorted list L_i containing all $w[1..1 + 2^i - 1]$, $w[2..2 + 2^i - 1]$, ..., $w[n..n + 2^i - 1]$.

Assume that we have the list L_i , and want to construct the list for $i + 1$. For each $j = 1, 2, \dots, n$ construct a pair $(\text{nr}_i(j), \text{nr}_i(j + 2^i))$, where $\text{nr}_i(j)$ is the position of $w[j..j + 2^i - 1]$ on the list L_i . Then sort all pairs, and notice that their order determines L_{i+1} .

Now let's try to improve the complexity to $\mathcal{O}(n \log n)$.

When you think about the previous solution, some fragments of the word will be compared again... and again...

We apply the *doubling* paradigm. To make the description easier, append n additional null characters to w , so it's actually a word of length $2n$.

$\mathcal{O}(n \log n)$ algorithm

For each $i = 0, 1, 2, \dots, \log n$ construct a sorted list L_i containing all $w[1..1 + 2^i - 1]$, $w[2..2 + 2^i - 1]$, ..., $w[n..n + 2^i - 1]$.

Assume that we have the list L_i , and want to construct the list for $i + 1$. For each $j = 1, 2, \dots, n$ construct a pair $(\text{nr}_i(j), \text{nr}_i(j + 2^i))$, where $\text{nr}_i(j)$ is the position of $w[j..j + 2^i - 1]$ on the list L_i . Then sort all pairs, and notice that their order determines L_{i+1} .

Now let's try to improve the complexity to $\mathcal{O}(n \log n)$.

When you think about the previous solution, some fragments of the word will be compared again... and again...

We apply the *doubling* paradigm. To make the description easier, append n additional null characters to w , so it's actually a word of length $2n$.

$\mathcal{O}(n \log n)$ algorithm

For each $i = 0, 1, 2, \dots, \log n$ construct a sorted list L_i containing all $w[1..1 + 2^i - 1]$, $w[2..2 + 2^i - 1]$, ..., $w[n..n + 2^i - 1]$.

Assume that we have the list L_i , and want to construct the list for $i + 1$. For each $j = 1, 2, \dots, n$ construct a pair $(nr_i(j), nr_i(j + 2^i))$, where $nr_i(j)$ is the position of $w[j..j + 2^i - 1]$ on the list L_i . Then sort all pairs, and notice that their order determines L_{i+1} .

Now let's try to improve the complexity to $\mathcal{O}(n \log n)$.

When you think about the previous solution, some fragments of the word will be compared again... and again...

We apply the *doubling* paradigm. To make the description easier, append n additional null characters to w , so it's actually a word of length $2n$.

$\mathcal{O}(n \log n)$ algorithm

For each $i = 0, 1, 2, \dots, \log n$ construct a sorted list L_i containing all $w[1..1 + 2^i - 1]$, $w[2..2 + 2^i - 1]$, ..., $w[n..n + 2^i - 1]$.

Assume that we have the list L_i , and want to construct the list for $i + 1$. For each $j = 1, 2, \dots, n$ construct a pair $(\text{nr}_i(j), \text{nr}_i(j + 2^i))$, where $\text{nr}_i(j)$ is the position of $w[j..j + 2^i - 1]$ on the list L_i . Then sort all pairs, and notice that their order determines L_{i+1} .

Now we are ready for the **real thing**.

Kärkkäinen and Sanders 2003

Suffix array can be constructed in $\mathcal{O}(n)$ time.

The idea is a recursive one. We will try to design the algorithm so that its running time can be expressed as $T(n) = T(\alpha n) + \mathcal{O}(n)$, where α is *some* constant less than 1.

Notice that the recursion solves to $T(n) = \mathcal{O}(n)$.

Now we are ready for the **real thing**.

Kärkkäinen and Sanders 2003

Suffix array can be constructed in $\mathcal{O}(n)$ time.

The idea is a recursive one. We will try to design the algorithm so that its running time can be expressed as $T(n) = T(\alpha n) + \mathcal{O}(n)$, where α is *some* constant less than 1.

Notice that the recursion solves to $T(n) = \mathcal{O}(n)$.

Now we are ready for the **real thing**.

Kärkkäinen and Sanders 2003

Suffix array can be constructed in $\mathcal{O}(n)$ time.

The idea is a recursive one. We will try to design the algorithm so that its running time can be expressed as $T(n) = T(\alpha n) + \mathcal{O}(n)$, where α is *some* constant less than 1.

Notice that the recursion solves to $T(n) = \mathcal{O}(n)$.

Now we are ready for the **real thing**.

Kärkkäinen and Sanders 2003

Suffix array can be constructed in $\mathcal{O}(n)$ time.

The idea is a recursive one. We will try to design the algorithm so that its running time can be expressed as $T(n) = T(\alpha n) + \mathcal{O}(n)$, where α is *some* constant less than 1.

Notice that the recursion solves to $T(n) = \mathcal{O}(n)$.

We partition all suffixes into three groups.

$$S_0 = \{w[3..n], w[6..n], w[9..n], \dots\}$$

$$S_1 = \{w[1..n], w[4..n], w[7..n], \dots\}$$

$$S_2 = \{w[2..n], w[5..n], w[8..n], \dots\}$$

S_r are all suffixes that start at positions of the form $3k + r$.

The goal is to sort all suffixes. We could start with something simpler: sorting (separately) each S_r .

We partition all suffixes into three groups.

$$S_0 = \{w[3..n], w[6..n], w[9..n], \dots\}$$

$$S_1 = \{w[1..n], w[4..n], w[7..n], \dots\}$$

$$S_2 = \{w[2..n], w[5..n], w[8..n], \dots\}$$

S_r are all suffixes that start at positions of the form $3k + r$.

The goal is to sort all suffixes. We could start with something simpler: sorting (separately) each S_r .

We partition all suffixes into three groups.

$$S_0 = \{w[3..n], w[6..n], w[9..n], \dots\}$$

$$S_1 = \{w[1..n], w[4..n], w[7..n], \dots\}$$

$$S_2 = \{w[2..n], w[5..n], w[8..n], \dots\}$$

S_r are all suffixes that start at positions of the form $3k + r$.

The goal is to sort all suffixes. We could start with something simpler: sorting (separately) each S_r .

We partition all suffixes into three groups.

$$S_0 = \{w[3..n], w[6..n], w[9..n], \dots\}$$

$$S_1 = \{w[1..n], w[4..n], w[7..n], \dots\}$$

$$S_2 = \{w[2..n], w[5..n], w[8..n], \dots\}$$

S_r are all suffixes that start at positions of the form $3k + r$.

The goal is to sort all suffixes. We could start with something simpler: sorting (separately) each S_r .

First trick

Say that we want to sort only S_1 . We could split w into blocks of length 3, treat each block as a single letter, and recursively solve a smaller problem of size $\frac{n}{3}$.

$w[1]$	$w[2]$	$w[3]$	$w[4]$	$w[5]$	$w[6]$	$\dots$	$w[n-2]$	$w[n-1]$	$w[n]$
--------	--------	--------	--------	--------	--------	---------	----------	----------	--------

We must be careful here: we promised that the input word of length $|w|$ will contain only letters $\{1, 2, \dots, |w|\}$, and here we create triples of letters.

Alphabet renaming

Create a sorted list of all triples $(w[i], w[i+1], w[i+2])$ and then compute the number $\text{nr}(i)$ of each triples there. This can be done in $\mathcal{O}(n)$ time using radix sort.

We append two null characters to w , so that the expression $(w[i], w[i+1], w[i+2])$ always makes sense.

$(w[1], w[2], w[3])$	$(w[4], w[5], w[6])$	$\dots$	$(w[n-2], w[n-1], w[n])$
----------------------	----------------------	---------	--------------------------

We must be careful here: we promised that the input word of length $|w|$ will contain only letters $\{1, 2, \dots, |w|\}$, and here we create triples of letters.

Alphabet renaming

Create a sorted list of all triples $(w[i], w[i+1], w[i+2])$ and then compute the number $\text{nr}(i)$ of each triples there. This can be done in $\mathcal{O}(n)$ time using radix sort.

We append two null characters to w , so that the expression $(w[i], w[i+1], w[i+2])$ always makes sense.

$\text{nr}(1)$	$\text{nr}(4)$	$\dots$	$\text{nr}(n - 2)$
----------------	----------------	---------	--------------------

We must be careful here: we promised that the input word of length $|w|$ will contain only letters $\{1, 2, \dots, |w|\}$, and here we create triples of letters.

Alphabet renaming

Create a sorted list of all triples $(w[i], w[i + 1], w[i + 2])$ and then compute the number $\text{nr}(i)$ of each triples there. This can be done in $\mathcal{O}(n)$ time using radix sort.

We append two null characters to w , so that the expression $(w[i], w[i + 1], w[i + 2])$ always makes sense.

$\text{nr}(1)$	$\text{nr}(4)$	$\dots$	$\text{nr}(n - 2)$
----------------	----------------	---------	--------------------

We must be careful here: we promised that the input word of length $|w|$ will contain only letters $\{1, 2, \dots, |w|\}$, and here we create triples of letters.

Alphabet renaming

Create a sorted list of all triples $(w[i], w[i + 1], w[i + 2])$ and then compute the number $\text{nr}(i)$ of each triples there. This can be done in $\mathcal{O}(n)$ time using radix sort.

We append two null characters to w , so that the expression $(w[i], w[i + 1], w[i + 2])$ always makes sense.

OK, so we can sort S_0 . Similarly, we can sort S_1 and S_2 , but we cannot afford to sort all of them!

Second trick

Assuming that we have already sorted $S_1 \cup S_2$, we can sort $S_0 \cup S_1 \cup S_2$ in $\mathcal{O}(n)$ time. For this we represent each suffix as a pair:

- $w[3k..n]$ becomes $(w[3k], w[3k + 1..n])$,
- $w[3k + 1..n]$ becomes $(w[3k + 1], w[3k + 2..n])$,
- $w[3k + 2..n]$ becomes $(w[3k + 2], w[3(k + 1)..n])$.

The order on pairs is clearly the same as order on suffixes.

OK, so we can sort S_0 . Similarly, we can sort S_1 and S_2 , but we cannot afford to sort all of them!

Second trick

Assuming that we have already sorted $S_1 \cup S_2$, we can sort $S_0 \cup S_1 \cup S_2$ in $\mathcal{O}(n)$ time. For this we represent each suffix as a pair:

- $w[3k..n]$ becomes $(w[3k], w[3k + 1..n])$,
- $w[3k + 1..n]$ becomes $(w[3k + 1], w[3k + 2..n])$,
- $w[3k + 2..n]$ becomes $(w[3k + 2], w[3(k + 1)..n])$.

The order on pairs is clearly the same as order on suffixes.

OK, so we can sort S_0 . Similarly, we can sort S_1 and S_2 , but we cannot afford to sort all of them!

Second trick

Assuming that we have already sorted $S_1 \cup S_2$, we can sort $S_0 \cup S_1 \cup S_2$ in $\mathcal{O}(n)$ time. For this we represent each suffix as a pair:

- $w[3k..n]$ becomes $(w[3k], w[3k + 1..n])$,
- $w[3k + 1..n]$ becomes $(w[3k + 1], w[3k + 2..n])$,
- $w[3k + 2..n]$ becomes $(w[3k + 2], w[3(k + 1)..n])$.

The order on pairs is clearly the same as order on suffixes.

OK, so we can sort S_0 . Similarly, we can sort S_1 and S_2 , but we cannot afford to sort all of them!

Second trick

Assuming that we have already sorted $S_1 \cup S_2$, we can sort $S_0 \cup S_1 \cup S_2$ in $\mathcal{O}(n)$ time. For this we represent each suffix as a pair:

- $w[3k..n]$ becomes $(w[3k], w[3k + 1..n])$,
- $w[3k + 1..n]$ becomes $(w[3k + 1], w[3k + 2..n])$,
- $w[3k + 2..n]$ becomes $(w[3k + 2], w[3(k + 1)..n])$.

The order on pairs is clearly the same as order on suffixes.

OK, so we can sort S_0 . Similarly, we can sort S_1 and S_2 , but we cannot afford to sort all of them!

Second trick

Assuming that we have already sorted $S_1 \cup S_2$, we can sort $S_0 \cup S_1 \cup S_2$ in $\mathcal{O}(n)$ time. For this we represent each suffix as a pair:

- $w[3k..n]$ becomes $(w[3k], w[3k + 1..n])$,
- $w[3k + 1..n]$ becomes $(w[3k + 1], w[3k + 2..n])$,
- $w[3k + 2..n]$ becomes $(w[3k + 2], w[3(k + 1)..n])$.

The order on pairs is clearly the same as order on suffixes.

OK, so we can sort S_0 . Similarly, we can sort S_1 and S_2 , but we cannot afford to sort all of them!

Second trick

Assuming that we have already sorted $S_1 \cup S_2$, we can sort $S_0 \cup S_1 \cup S_2$ in $\mathcal{O}(n)$ time. For this we represent each suffix as a pair:

- $w[3k..n]$ becomes $(w[3k], w[3k + 1..n])$,
- $w[3k + 1..n]$ becomes $(w[3k + 1], w[3k + 2..n])$,
- $w[3k + 2..n]$ becomes $(w[3k + 2], w[3(k + 1)..n])$.

The order on pairs is clearly the same as order on suffixes.

Why does this work?

We have already sorted $S_1 \cup S_2$.

We can easily sort S_0 by replacing each suffix $w[3k..n]$ with the corresponding pair, and then sorting the pairs using radix sort. Just replace the second element of each pair with its position in the already known sorted sequence of all $S_1 \cup S_2$!

Then we only have to merge two sorted sequences of length $\frac{1}{3}n$ and $\frac{2}{3}n$. This can be done in linear time, assuming that we can compare any two elements in constant time.

Why does this work?

We have already sorted $S_1 \cup S_2$.

We can easily sort S_0 by replacing each suffix $w[3k..n]$ with the corresponding pair, and then sorting the pairs using radix sort. Just replace the second element of each pair with its position in the already known sorted sequence of all $S_1 \cup S_2$!

Then we only have to merge two sorted sequences of length $\frac{1}{3}n$ and $\frac{2}{3}n$. This can be done in linear time, assuming that we can compare any two elements in constant time.

Why does this work?

We have already sorted $S_1 \cup S_2$.

We can easily sort S_0 by replacing each suffix $w[3k..n]$ with the corresponding pair, and then sorting the pairs using radix sort. Just replace the second element of each pair with its position in the already known sorted sequence of all $S_1 \cup S_2$!

Then we only have to merge two sorted sequences of length $\frac{1}{3}n$ and $\frac{2}{3}n$. This can be done in linear time, assuming that we can compare any two elements in constant time.

Why does this work?

We have already sorted $S_1 \cup S_2$.

We can easily sort S_0 by replacing each suffix $w[3k..n]$ with the corresponding pair, and then sorting the pairs using radix sort. Just replace the second element of each pair with its position in the already known sorted sequence of all $S_1 \cup S_2$!

Then we only have to merge two sorted sequences of length $\frac{1}{3}n$ and $\frac{2}{3}n$. This can be done in linear time, assuming that we can compare any two elements in constant time.

So, how to compare any $w[3i..n] \in S_0$ with $w[j..n] \in S_1 \cup S_2$?

- $j = 3k + 1$, then represent $w[3i..n]$ as $(w[3i], w[3i + 1..n])$ and $w[j..n] = w[3k + 1..n]$ as $(w[3k + 1], w[3k + 2..n])$. $w[3i + 1..n]$ can be compared with $w[3k + 2]$ as they both belong to $S_1 \cup S_2$.
- $j = 3k + 2$, then represent $w[3i..n]$ as a **triple** $(w[3i], w[3i + 1], w[3i + 2..n])$ and $w[j..n] = w[3k + 2..n]$ as $(w[3k + 2], w[3(k + 1)], w[3(k + 1) + 1..n])$. $w[3i + 2..n]$ can be compared with $w[3(k + 1) + 1]$ as they both belong to $S_1 \cup S_2$.

Hence any two elements can be compared in constant time.

So, how to compare any $w[3i..n] \in S_0$ with $w[j..n] \in S_1 \cup S_2$?

- $j = 3k + 1$, then represent $w[3i..n]$ as $(w[3i], w[3i + 1..n])$ and $w[j..n] = w[3k + 1..n]$ as $(w[3k + 1], w[3k + 2..n])$. $w[3i + 1..n]$ can be compared with $w[3k + 2]$ as they both belong to $S_1 \cup S_2$.
- $j = 3k + 2$, then represent $w[3i..n]$ as a **triple** $(w[3i], w[3i + 1], w[3i + 2..n])$ and $w[j..n] = w[3k + 2..n]$ as $(w[3k + 2], w[3(k + 1)], w[3(k + 1) + 1..n])$. $w[3i + 2..n]$ can be compared with $w[3(k + 1) + 1]$ as they both belong to $S_1 \cup S_2$.

Hence any two elements can be compared in constant time.

So, how to compare any $w[3i..n] \in S_0$ with $w[j..n] \in S_1 \cup S_2$?

- $j = 3k + 1$, then represent $w[3i..n]$ as $(w[3i], w[3i + 1..n])$ and $w[j..n] = w[3k + 1..n]$ as $(w[3k + 1], w[3k + 2..n])$. $w[3i + 1..n]$ can be compared with $w[3k + 2]$ as they both belong to $S_1 \cup S_2$.
- $j = 3k + 2$, then represent $w[3i..n]$ as a **triple** $(w[3i], w[3i + 1], w[3i + 2..n])$ and $w[j..n] = w[3k + 2..n]$ as $(w[3k + 2], w[3(k + 1)], w[3(k + 1) + 1..n])$. $w[3i + 2..n]$ can be compared with $w[3(k + 1) + 1]$ as they both belong to $S_1 \cup S_2$.

Hence any two elements can be compared in constant time.

So, how to compare any $w[3i..n] \in S_0$ with $w[j..n] \in S_1 \cup S_2$?

- $j = 3k + 1$, then represent $w[3i..n]$ as $(w[3i], w[3i + 1..n])$ and $w[j..n] = w[3k + 1..n]$ as $(w[3k + 1], w[3k + 2..n])$. $w[3i + 1..n]$ can be compared with $w[3k + 2]$ as they both belong to $S_1 \cup S_2$.
- $j = 3k + 2$, then represent $w[3i..n]$ as a **triple** $(w[3i], w[3i + 1], w[3i + 2..n])$ and $w[j..n] = w[3k + 2..n]$ as $(w[3k + 2], w[3(k + 1)], w[3(k + 1) + 1..n])$. $w[3i + 2..n]$ can be compared with $w[3(k + 1) + 1]$ as they both belong to $S_1 \cup S_2$.

Hence any two elements can be compared in constant time.

We are almost done. The only remaining question is how to sort $S_1 \cup S_2$. We know how to sort S_1 and S_2 , but we need a stronger observation.

Third trick

The order on all $S_1 \cup S_2$ can be computed by sorting all suffixes of $w' = \text{nr}(1)\text{nr}(4)\text{nr}(7) \dots \text{nr}(2)\text{nr}(5)\text{nr}(8) \dots$

Finally, we get an algorithm with the running time of the form $T(n) = T(\frac{2}{3}n) + \mathcal{O}(n)$, which is $\mathcal{O}(n)$.



We are almost done. The only remaining question is how to sort $S_1 \cup S_2$. We know how to sort S_1 and S_2 , but we need a stronger observation.

Third trick

The order on all $S_1 \cup S_2$ can be computed by sorting all suffixes of $w' = \text{nr}(1)\text{nr}(4)\text{nr}(7) \dots \text{nr}(2)\text{nr}(5)\text{nr}(8) \dots$

Finally, we get an algorithm with the running time of the form $T(n) = T(\frac{2}{3}n) + \mathcal{O}(n)$, which is $\mathcal{O}(n)$.



We are almost done. The only remaining question is how to sort $S_1 \cup S_2$. We know how to sort S_1 and S_2 , but we need a stronger observation.

Third trick

The order on all $S_1 \cup S_2$ can be computed by sorting all suffixes of $w' = \text{nr}(1)\text{nr}(4)\text{nr}(7) \dots \text{nr}(2)\text{nr}(5)\text{nr}(8) \dots$

Finally, we get an algorithm with the running time of the form $T(n) = T(\frac{2}{3}n) + \mathcal{O}(n)$, which is $\mathcal{O}(n)$.



We are almost done. The only remaining question is how to sort $S_1 \cup S_2$. We know how to sort S_1 and S_2 , but we need a stronger observation.

Third trick

The order on all $S_1 \cup S_2$ can be computed by sorting all suffixes of $w' = \text{nr}(1)\text{nr}(4)\text{nr}(7) \dots \text{nr}(2)\text{nr}(5)\text{nr}(8) \dots$

Finally, we get an algorithm with the running time of the form $T(n) = T(\frac{2}{3}n) + \mathcal{O}(n)$, which is $\mathcal{O}(n)$.

